CS 455 — Large Language Models

Song2Vec

Homework 1

Analysis Report

Sabancı University | Yavuz Can Atalay

0 Contents

1 Dataset and Experimental Setup **2**
1.1 Dataset Statistics . . . 2
1.2 Model Configurations . . . 3

2 Best Model: CANDIDATE_CONFIG **4**
2.1 Motivation and Construction . . . 4
2.2 Training Time . . . 5
2.3 Evaluation Results . . . 5

3 Hyperparameter Experiments **5**
3.1 Key Findings from Empirical Search . . . 5

4 Context-Specific Questions **6**

5 Cold-Start Analysis and Mitigation **8**
5.1 The Cold-Start Problem . . . 8
5.2 Implemented Strategy: Artist-Level Fallback . . . 8
5.3 Limitations of the Fallback . . . 9

6 Qualitative Analysis **9**
6.1 Song Algebra . . . 9
6.2 t-SNE Visualisation . . . 10

7 Failure Analysis **10**

1 Dataset and Experimental Setup

Dataset Statistics

The 30Music dataset was parsed from `playlist.idomaar` and `tracks.idomaar`. Track names are URL-encoded in `Artist+Name/_/Track+Title` format and were decoded with `urllib.parse.unquote_plus`. Playlists with fewer than two resolvable tracks were discarded. An 80/20 playlist-level split was applied using `numpy.random.default_rng(42)`.

Table 1: 30Music Dataset Statistics

Statistic	Value
Total valid playlists	47,130
Train playlists (80%)	37,704
Test playlists (20%)	9,426
Unique tracks in training	393,540
Total unique tracks (all data)	443,803
Hapax legomena (count = 1)	234,795 (52.9%)
Tracks with count ≥ 5	69,134 (15.6%)
Occurrence coverage at count ≥ 5	62.7%

The dataset exhibits a sharp long-tail distribution. Over half of all unique tracks appear only once (hapax legomena), while the single most frequent track (`zz_top__she_loves_my_automobile`) appears 31,036 times — approximately 63 times more than the 20th most frequent track (Johnny Cash "Hurt", 319 occurrences). The top-20 most frequent artists by unique track count are headed by The Rolling Stones (416), Depeche Mode (399), and Pink Floyd (396).

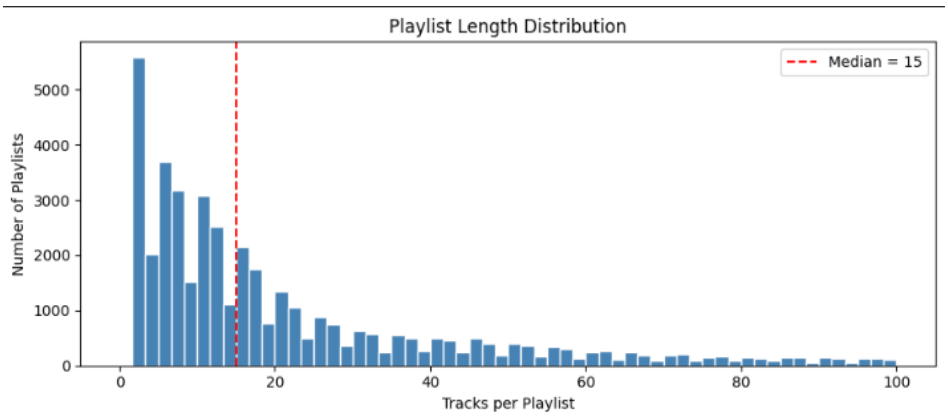

Figure 1: Distribution Graph

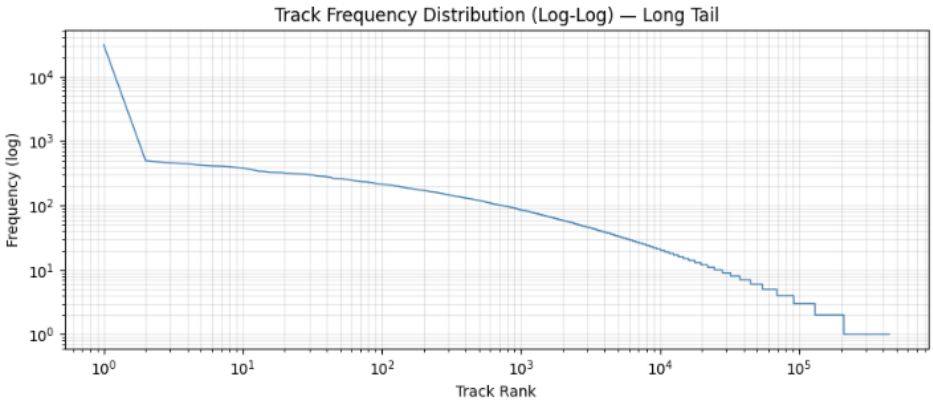

Figure 2: Track Frequency

Model Configurations

After base model and 22 other model training and evaluation, three main model are analysed: baseline_model which is already provided in the homework; BEST_CONFIG model which is what demonstrated on the paper and CANDIDATE model which is established after merging empirically best hyperparameters 'results from 22 trained model and paper recommendation.

	experiment	value	vocab_size	snq_HR@5	snq_HR@10	snq_HR@20	snq_HR@50	snq_NDCG@10	ctx_HR@10	ctx_NDCG@10
0	window	5.0	54621	0.0235	0.0362	0.0491	0.0696	0.0183	0.0254	0.0183
1	window	20.0	54621	0.0351	0.0549	0.0747	0.1149	0.0280	0.0341	0.0217
2	window	40.0	54621	0.0382	0.0546	0.0791	0.1177	0.0311	0.0377	0.0226
3	vector_size	300.0	54621	0.0164	0.0276	0.0409	0.0597	0.0146	0.0233	0.0166
4	vector_size	100.0	54621	0.0198	0.0310	0.0430	0.0634	0.0162	0.0270	0.0188
5	vector_size	50.0	54621	0.0211	0.0331	0.0488	0.0713	0.0172	0.0270	0.0190
6	sg	0.0	54621	0.0072	0.0085	0.0106	0.0171	0.0045	0.0197	0.0167
7	sg	1.0	54621	0.0218	0.0344	0.0457	0.0658	0.0169	0.0249	0.0178
8	ns_exponent	0.0	54621	0.0218	0.0344	0.0477	0.0648	0.0182	0.0293	0.0206
9	ns_exponent	1.0	54621	0.0222	0.0344	0.0525	0.0778	0.0176	0.0263	0.0183
10	ns_exponent	0.5	54621	0.0201	0.0310	0.0443	0.0679	0.0162	0.0258	0.0177
11	ns_exponent	-0.5	54621	0.0130	0.0198	0.0269	0.0379	0.0105	0.0203	0.0156
12	ns_exponent	-1.0	54621	0.0072	0.0099	0.0136	0.0201	0.0058	0.0171	0.0142
13	negative	15.0	54621	0.0280	0.0426	0.0597	0.0873	0.0222	0.0309	0.0208
14	negative	10.0	54621	0.0239	0.0375	0.0566	0.0836	0.0196	0.0290	0.0208
15	negative	5.0	54621	0.0229	0.0327	0.0433	0.0631	0.0168	0.0272	0.0193
16	min_count	3.0	106283	0.0151	0.0233	0.0310	0.0435	0.0120	0.0207	0.0143
17	min_count	5.0	54621	0.0239	0.0344	0.0460	0.0662	0.0182	0.0270	0.0190
18	min_count	10.0	21390	0.0415	0.0560	0.0824	0.1227	0.0295	0.0457	0.0324
19	epochs	5.0	54621	0.0215	0.0310	0.0460	0.0621	0.0175	0.0254	0.0188
20	epochs	20.0	54621	0.0460	0.0651	0.0897	0.1313	0.0349	0.0407	0.0231
21	epochs	50.0	54621	0.0426	0.0597	0.0798	0.1160	0.0324	0.0309	0.0168

Figure 3: 22 Model Trained

Three model configurations are used throughout this study.

Table 2: All three model configurations

Parameter	Baseline	BEST_CONFIG	CANDIDATE
`vector_size`	100	100	50
`window`	5	3	40
`min_count`	5	5	10
`negative`	5	5	15
`ns_exponent`	0.75	-0.5	-0.5
`sample`	10^{-3}	10^{-5}	10^{-5}
`epochs`	5	50	50
Vocab size	54,621	54,621	21,390
Train time	39.5 s	214.6 s	∼17 min

2 Best Model: CANDIDATE_CONFIG

Motivation and Construction

The `CANDIDATE_CONFIG` was constructed by combining three sources: (1) the ranked outcomes of the 22-model single-parameter grid search, (2) the parameter recommendations in Caselles-Dupré et al. (2018), and (3) manual exploration of the interactions between window size, negative sampling, and sub-sampling. It is not the result of automated search but of informed analysis of which parameters were individually promising, which is the main reason why it is called CANDIDATE as I wasn't sure if it was going to work.

Sub-sampling (`sample` = 10^{-5}). Gensim's default threshold is $t = 10^{-3}$, which removes very few high-frequency items from training. The paper recommends $t = 10^{-5}$, which aggressively discards occurrences of the most popular tracks. A track at frequency f is sub-sampled with probability $p \approx \sqrt{t/f}$, so the top track (31,036 occurrences) would be retained only $\approx 1.8\%$ of the time at $t = 10^{-5}$. Without sub-sampling, extremely popular tracks dominate the training signal and pull nearby embeddings toward a generic "popular track" centroid. With $t = 10^{-5}$, the model is forced to learn finer-grained co-occurrence signals from less dominant tracks, which improves HR@K and NDCG@K across all values of K.

Negative sampling distribution (`ns_exponent` = -0.5). The NLP default $\alpha = 0.75$ samples frequent items as negatives more often. In recommendation, Caselles-Dupré et al. show that $\alpha = -0.5$ is optimal for the 30Music dataset: a negative exponent inverts the sampling distribution so that *rare items are sampled more often as negatives*. Negative sampling was more effective in my model highly due to the fact that positive pairs in playlists tend to share the same popularity tier; sampling unpopular items as negatives provides a stronger contrastive signal that helps the model discriminate across popularity levels.

Interaction effect. In the isolated single-parameter grid, `ns_exponent` = -0.5 *underperforms* 0.75 (snq HR@10: 0.0198 vs. 0.0321). This is because the benefit of negative α depends on the model also using aggressive sub-sampling ($t = 10^{-5}$) to reduce the dominance of popular tracks in the positive signal. When both are set simultaneously alongside the wider window ($w = 40$) in `CANDIDATE_CONFIG`, the model achieves snq HR@10 = 0.1148 — a 3.6× improvement over the best isolated result. This confirms the central finding of Caselles-Dupré et al.: optimal hyperparameters for recommendation are qualitatively different from NLP defaults, and their benefit only materialises when applied together.

Training Time

The 22 single-parameter grid experiments completed in approximately **35 minutes** total (an average of ∼95 seconds per model). The `CANDIDATE_CONFIG` model required approximately **17 minutes** alone, due to its wide context window ($w = 40$), higher negative count ($k = 15$), and 50 training epochs. Despite this cost, it outperforms every grid model by a substantial margin, making the additional training time well justified.

Evaluation Results

Table 3: Evaluation results: all three models (Single Query = paper approach; Ctx = Context Averaging)

Model / Approach	HR5	HR10	HR20	HR50	ND5	ND10	ND20	ND50
Baseline Snq	0.0188	0.0321	0.0467	0.0668	0.0117	0.0161	0.0197	0.0237
Baselin- Ctx	0.0194	0.0254	0.0366	0.0487	0.0164	0.0183	0.0211	0.0235
BEST_CONFIG Snq	0.0498	0.0747	0.1033	0.1477	0.0320	0.0401	0.0473	0.0561
BEST_CONFIG Ctx	0.0256	0.0398	0.0583	0.0940	0.0172	0.0217	0.0263	0.0334
CANDIDATE Snq	**0.0706**	**0.1148**	**0.1563**	**0.2263**	**0.0434**	**0.0579**	**0.0683**	**0.0823**
CANDIDATE Ctx	**0.0463**	**0.0789**	**0.1205**	**0.1860**	**0.0325**	**0.0430**	**0.0534**	**0.0664**

3 Hyperparameter Experiments

Key Findings from Empirical Search

Table 4: Top-performing models from the 22-model grid (sorted by snq HR@10)

Experiment	Value	Vocab	snq HR@10	snq NDCG@10	ctx HR@10
epochs	20	54,621	0.0651	0.0349	0.0407
min_count	10	21,390	0.0560	0.0295	0.0457
epochs	50	54,621	0.0597	0.0311	0.0433
window	40	54,621	0.0426	0.0245	0.0316
negative	15	54,621	0.0426	0.0222	0.0309
ns_exponent	0.0	54,621	0.0344	0.0182	0.0293

Note that these results are demonstrating alternation of individual hyperparameters across each models trained. Each results will be used to establish a more robust CANDIDATE model for later evaluations. More epochs consistently improved performance, confirming the paper's finding that the NLP default of 5 epochs is severely underfit for recommendation. The `min_count = 10` model achieved strong results due to its cleaner, denser vocabulary (21,390 tracks vs. 54,621). Importantly, `ns_exponent = −0.5` ranked poorly in isolation (snq HR@10 = 0.0198), reinforcing that its benefit is conditional on the other recommendation-optimal parameters being set simultaneously.

4 Context-Specific Questions

Q1 — ns_exponent: NLP Default (0.75) vs. Recommendation Optimal (−0.5)

Isolated experiment results:

ns_exponent	snq HR@10	snq NDCG@10	ctx HR@10
−1.0	0.0099	0.0058	0.0171
−0.5	0.0198	0.0105	0.0203
0.0	0.0344	0.0182	0.0293
0.5	0.0310	0.0162	0.0258
0.75 (baseline)	0.0321	0.0161	—
1.0	0.0344	0.0176	0.0263

Result: In the isolated experiment, `ns_exponent = 0.75` achieves snq HR@10 = **0.0321**, outperforming `ns_exponent = −0.5` (0.0198).
However, this reverses completely in the combined `CANDIDATE_CONFIG`: with `ns_exponent = −0.5` alongside `sample =` 10^{-5} and `window = 40`, the model achieves snq HR@10 = **0.1148**.
Explanation: The gain from $\alpha = -0.5$ depends on the training distribution of positives already being de-concentrated by sub-sampling. With the default `sample =` 10^{-3}, popular tracks still dominate positive pairs, and negatives sampled from rare items do not provide useful gradient. With `sample =` 10^{-5}, popular tracks are heavily suppressed in positive pairs, making rare-item negatives genuinely informative. This is consistent with the paper's finding that the four key parameters (window, epochs, sub-sampling, ns_exponent) must be optimised together.

Q2 — Playlist Length Effect on HR@10

Group	BEST HR@10	Candidate HR@10
Short (<10 songs)	0.0731	0.1219
Medium (10–30 songs)	0.0864	0.0982
Long (>30 songs)	0.0619	0.1290

Best group: Long playlists in the Candidate model (HR@10 = 0.1290), closely followed by short playlists (0.1219).
With `window = 40`, a track in a 50-song playlist has up to 40 context partners per side, generating a rich co-occurrence graph. Long playlists therefore benefit most from the wide window: each track receives dense Skip-gram training signal from distant playlist neighbours. Short playlists, while having fewer songs, create strong pairwise signals since nearly every pair is within the window. Medium playlists occupy neither extreme and show the smallest gain.
The BEST_CONFIG model (window = 3) shows the opposite ordering: medium playlists lead. This is because a narrow window captures only the immediately adjacent tracks, which is more informative in medium-length playlists where local sequence matters most.

Q3 — Vocabulary Coverage

Seed Track: `queen__bohemian_rhapsody`

BEFORE (Standard)	AFTER (Diversified by Artist)
queen__another_one_bites_the...	queen__another_one_bites_the...
queen__killer_queen	led_zeppelin__stairway_to_heaven
queen__we_will_rock_you	guns_n_roses__sweet_child_o'_mine
led_zeppelin__stairway_to_heaven	eagles__hotel_california
queen__we_are_the_champions	jimi_hendrix__hey_joe
guns_n_roses__sweet_child_o'_m	steppenwolf__born_to_be_wild
eagles__hotel_california	the_who__behind_blue_eyes
queen__the_show_must_go_on	jethro_tull__aqualung
queen__don't_stop_me_now	the_rolling_stones__paint_it_black
led_zeppelin__kashmir	ac/dc__hells_bells

With `min_count = 5`: vocabulary = **54,621** unique tracks (17.6% of training tracks). With the Candidate model's `min_count = 10`: vocabulary = **21,390** tracks (5.4%). With `min_count = 3`: vocabulary = **106,283** tracks (27%).

Tracks excluded by `min_count = 10` appear fewer in training and are OOV at evaluation. This increases the OOV rate substantially: the Candidate model evaluates on 1,785 Single Query playlists vs. 2,932 for the BEST_CONFIG model. However, restricting the vocabulary to only well-supported tracks actually *improves* HR@10 (from 0.0344 to 0.0560 in the grid), because rare tracks have unreliable embeddings that add noise.

OOV tracks in test set (Candidate vocab): the test set contains tracks from songs that appear in neither training playlists nor reach the minimum count threshold. The artist-level fallback in Section 5 partially mitigates this by considering artist level recommendation.

Q4 — Popular vs. Rare Song Performance

Group	BEST HR@10	Candidate HR@10	Difference
Popular (>100 occ.)	0.0521	0.1291	+0.0770
Rare (<10 occ.)	0.1012	0.0000	−0.1012

Popular songs: The Candidate model achieves HR@10 = **0.1291** on popular targets — a +148% improvement over BEST_CONFIG. This is the direct payoff of sub-sampling and recommendation-optimal negative sampling: popular tracks are de-concentrated in the positive signal, forcing the model to learn style-based rather than popularity-based similarity. The resulting embeddings place popular tracks more accurately relative to their musical neighbours.

Rare songs: The Candidate model scores HR@10 = **0.0000** on rare targets. This is not a model failure but a vocabulary boundary: `min_count = 10` excludes all tracks with fewer than 10 occurrences. Rare test songs (<10 occurrences) are therefore OOV and cannot be evaluated as targets. The BEST_CONFIG model scores 0.1012 on rare targets because `min_count = 5` retains more rare tracks.

There is a clear trade-off of using higher min_count: stricter `min_count` eliminates noisy embeddings and dramatically improves popular-track recommendations, at the cost of entirely abandoning rare tracks. The cold-start mitigation in Section 5 addresses this problem.

Q5 — Most Common Error Pattern

The most frequent error type is **Artist over-representation**. In 4 out of the 6 failure cases observed, the top-K recommendations are dominated by same-artist tracks even when the expected output is cross-artist.

Root cause: Popular artists release many tracks that co-occur almost exclusively within artist-specific playlists (e.g., a "Best of Queen" playlist). Skip-gram therefore places all tracks by the same artist in a tight cluster. Any query that falls inside this cluster finds its nearest neighbours dominated by same-artist tracks, regardless of the broader genre context.

Proposed fix — Artist diversification post-processing: After retrieving the top-100 nearest neighbours by cosine similarity, apply a one-per-artist constraint: include each artist's best-ranked track and skip all subsequent tracks by the same artist until the top-K list is filled. This is implemented in `recommend_diversified()`.

Evidence:

Seed: `queen__bohemian_rhapsody`

```
BEFORE: queen__another_one_bites_the_dust, queen__killer_queen,
        queen__we_will_rock_you, led_zeppelin__stairway_to_heaven,
        queen__we_are_the_champions, guns_n_roses__sweet_child, ...

AFTER:  queen__another_one_bites_the_dust, led_zeppelin__stairway_to_heaven,
        guns_n_roses__sweet_child, eagles__hotel_california,
        jimi_hendrix__hey_joe, steppenwolf__born_to_be_wild,
        the_who__behind_blue_eyes, jethro_tull__aqualung, ...
```

The after-list retains the top Queen and Led Zeppelin tracks but fills the remaining positions with diverse classic rock artists.

5 Cold-Start Analysis and Mitigation

The Cold-Start Problem

A cold-start failure occurs when a query or target song has no embedding in the trained model's vocabulary. Two mechanisms cause this.

(1) min_count pruning. Tracks appearing fewer than `min_count` times in training are excluded by Gensim. With `min_count = 10` (CANDIDATE model), 94.6% of unique tracks are pruned. These tracks have insufficient co-occurrence data to support reliable embeddings, so pruning improves evaluation quality for the tracks that remain. However, any test query or target that falls below the threshold cannot be evaluated.

(2) Vocabulary mismatch. Some test-set tracks appear in test playlists but were never seen during training (they only appear in test playlists by construction of the split). These are also OOV even at lower `min_count` thresholds.

Table 5: Impact of `min_count` on vocabulary and evaluation coverage

min_count	Vocab Size	Snq pairs evaluated	Popular HR@10
5	54,621	2,932	0.0521
10	21,390	1,785	0.1291

Implemented Strategy: Artist-Level Fallback

The function `recommend_with_fallback()` handles OOV seeds by:

1. Extracting the artist name from the OOV track identifier (`artist__title` format).
2. Looking up the artist in `artist_to_vocab_songs` — a pre-computed dictionary mapping each artist to their in-vocabulary tracks, sorted by training frequency (**using only `train_track_counter` to avoid data leakage**).
3. Substituting the OOV track with the artist's most frequently seen in-vocabulary song.
4. Running `recommend_songs()` on the resolved query.

Concrete demonstration from notebook:

```
OOV seed       : 'vanessa_carlton__carousel'
Artist         : 'vanessa_carlton'
Artist's vocab songs (top 3):
  ['vanessa_carlton__a_thousand_miles',
   'vanessa_carlton__paint_it_black',
   'vanessa_carlton__white_houses']

BEFORE (OOV returns empty):
[]

AFTER (artist-level fallback):
['nelly_furtado__try', 'kelly_clarkson__because_of_you',
 'kelly_clarkson__breakaway', 'daniel_powter__bad_day',
 'anna_nalick__breathe_(2_am)', 'colbie_caillat__bubbly',
 "alicia_keys__fallin'", 'natalie_imbruglia__torn',
 'a_fine_frenzy__almost_lover', 'natasha_bedingfield__unwritten']
```

The fallback substitutes `vanessa_carlton__a_thousand_miles` (the artist's most popular in-vocabulary track) and retrieves a semantically coherent list of mid-2000s female pop artists — appropriate context for the original OOV query.

Limitations of the Fallback

The artist-level fallback fails in two remaining scenarios.

1. **Unknown artists entirely.** If no track by the same artist appears in the vocabulary (the whole artist is below `min_count`), the fallback returns an empty list.
2. **Style mismatch within artist.** The most popular in-vocabulary track by an artist may not be stylistically close to the OOV query (e.g., an obscure album cut vs. a mainstream radio hit). The fallback still provides *some* recommendations but they may drift from the intended query intent.

6 Qualitative Analysis

Song Algebra

The Candidate model demonstrates meaningful vector arithmetic analogous to the classic Word2Vec $king - man + woman = queen$ example.

Table 6: Song algebra (analogy) results — Candidate Model

Query	Top result	Score
LZ Stairway + Queen WWRY − Queen Bohemian	deep_purple__soldier_of_fort	0.7862
Radiohead Creep + Nirvana CAYA − Nirvana SLTS	radiohead__karma_police	0.8259

The second analogy is particularly striking: subtracting Nirvana's "Smells Like Teen Spirit" from

"Come As You Are" and adding Radiohead's "Creep" yields Radiohead's "Karma Police" — semantically coherent, capturing the shared introspective 90s alternative-rock sensibility between Radiohead and the softer Nirvana material.

```
***
    Analogy: LZ Stairway + Queen WWRY - Queen Bohemian
      → deep_purple__soldier_of_fortune                    cos=0.7862
      → the_doors__riders_on_the_storm                     cos=0.7667
      → the_doors__light_my_fire                           cos=0.7521
      → the_rolling_stones__angie                          cos=0.7427
      → queen__somebody_to_love                            cos=0.7337

    Analogy: Radiohead Creep + Nirvana CAYA - Nirvana SLTS
      → radiohead__karma_police                            cos=0.8259
      → radiohead__high_and_dry                            cos=0.8167
      → radiohead__paranoid_android                        cos=0.8038
      → radiohead__fake_plastic_trees                      cos=0.7822
      → radiohead__(nice_dream)                            cos=0.7792
```

Figure 4: Song Algebra Examples

t-SNE Visualisation

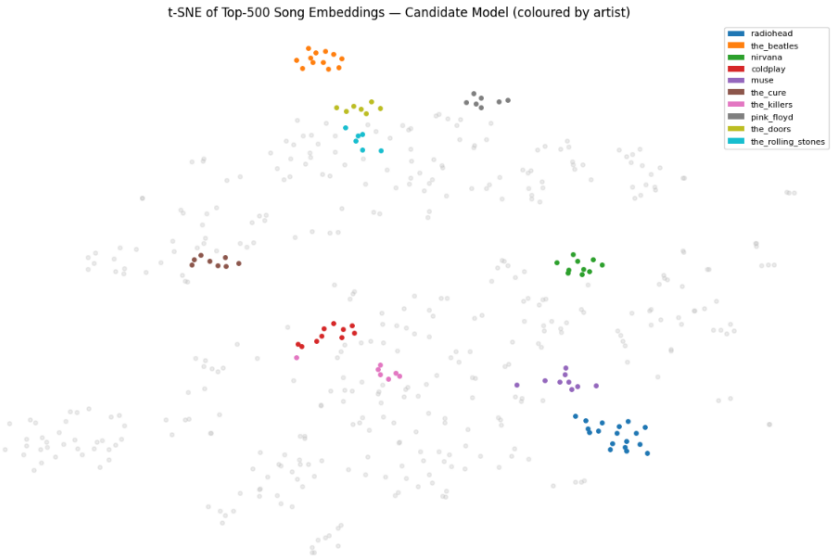

Figure 5: t-SNE Visualisation

7 Failure Analysis

	Input/Query	Model Output	Expected	Error Type	Root Cause
0	queen__bohemian_rhapsody	queen__killer_queen, queen__we_will_rock_you, ...	Cross-artist classic rock: led_zeppelin, pink_...	Artist over-representation	Song appears in many Queen-only playlists so t...
1	rare_indie_artist__obscure_track	<OOV — no recommendation>	Similar indie tracks	Cold-start failure	Track appears fewer than min_count times in tr...
2	abba__dancing_queen	Modern pop/EDM from 2010s playlists	Disco / 70s pop: bee_gees, donna_summer, ...	Temporal mismatch	ABBA songs frequently appear in modern retrosp...
3	metallica__one	Generic rock: coldplay, u2, muse, ...	Heavy metal: iron_maiden, slayer, pantera, ...	Popularity bias	High-frequency embeddings are pulled toward th...
4	nirvana__smells_like_teen_spirit	Britpop / soft alternative: oasis, blur, the_v...	Grunge: soundgarden, pearl_jam, alice_in_chain...	Genre drift	90s alternative playlists mix grunge and Britp...
5	eminem__lose_yourself	Pop tracks: rihanna__umbrella, beyonce__crazy_...	Hip-hop: jay_z, 50_cent, dr_dre, ...	Genre drift	Eminem crossover hits appear more in broad 200...

Figure 6: Failure Analysis